

Project: Prism IQ

Version: 1.0

Status: Mandatory

Applies To: Backend, Frontend, ML, Utilities & Tests

1. Engineering Philosophy

Every line of code should be:

- Readable
- Predictable
- Testable
- Reusable
- Maintainable

Code is written for **future developers**, not just for the compiler.

2. General Principles

Always Prefer

- ✓ Simplicity
 - ✓ Explicitness
 - ✓ Readability
 - ✓ Type Safety
 - ✓ Reusable Components
-

Avoid

- ✗ Clever hacks
 - ✗ Duplicate logic
 - ✗ Magic numbers
 - ✗ Hardcoded strings
 - ✗ Giant functions
 - ✗ Circular dependencies
-

3. Project Folder Rules

Every folder must have one clear responsibility.

Example:

```
backend/  
  
  api/  
  services/  
  repositories/  
  models/  
  schemas/  
  core/  
  utils/  
  ml/  
  reports/  
  tests/
```

Never mix business logic inside API routes.

4. File Naming

Python

```
snake_case.py
```

Example

```
dataset_service.py  
report_generator.py
```

React

```
PascalCase.tsx
```

Example

```
Dashboard.tsx  
UploadCard.tsx  
InsightsPanel.tsx
```

Hooks

```
useUpload.ts
```

Utilities

```
file_utils.py
```

```
date_utils.py
```

5. Naming Rules

Variables

```
dataset_name  
missing_count
```

Functions

```
generate_profile()  
  
clean_dataset()  
  
train_models()
```

Classes

```
DatasetService  
  
ReportGenerator  
  
MLTrainer
```

Constants

```
MAX_FILE_SIZE  
  
SUPPORTED_FORMATS  
  
DEFAULT_RANDOM_STATE
```

6. Function Standards

Every function should do **one thing only**.

Good

```
clean_missing_values()
```

Bad

```
clean_dataset_generate_report_train_model()
```

Maximum recommended function length

```
40–60 lines
```

Split larger logic into helper functions.

7. Docstrings

Every public function must include documentation.

Example

```
def generate_profile(dataset_id: UUID):  
    """  
    Generate dataset profiling statistics.  
  
    Args:  
        dataset_id: Dataset identifier.  
  
    Returns:  
        ProfileResponse object.  
    """
```

8. Type Hints

Always required.

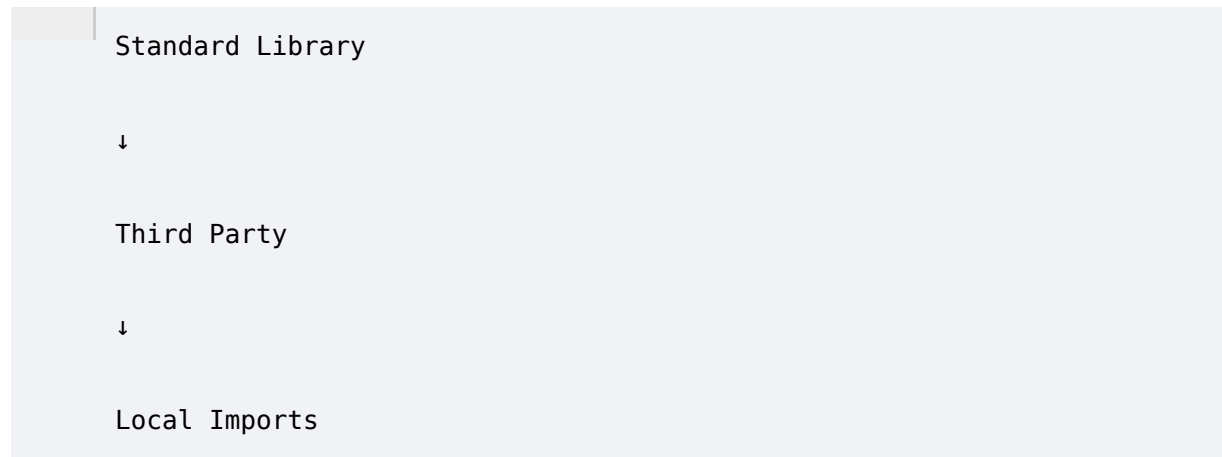
Example

```
def train_models(  
    df: pd.DataFrame,  
    target: str  
    ) -> ModelResult:
```

Never omit return types.

9. Imports

Order



Example

```
import os

import pandas as pd

from app.models import Dataset
```

Never use wildcard imports.

10. Comments

Explain **why**.

Not **what**.

Bad

```
# increment i

i += 1
```

Good

```
# Skip invalid rows to preserve row alignment
```

11. Logging

Never

```
print()
```

Use

```
logger.info()
```

```
logger.warning()
```

```
logger.error()
```

```
logger.exception()
```

Every major operation should be logged.

12. Exceptions

Never

```
except:
```

Always

```
except ValueError:
```

```
except HTTPException:
```

Raise meaningful errors.

13. API Routes

Routes should only:

- Validate request
- Call service
- Return response

Never write business logic in routes.

Example

```
router.post(...)
```

↓

```
service.generate()
```

↓

```
repository.save()
```

14. Service Layer

Services coordinate:

- Repository
- ML
- Reports
- Cleaning
- External integrations

Services contain business logic.

15. Repository Layer

Repositories only:

- Query database
- Save records
- Update records
- Delete records

Nothing else.

16. Database Rules

Never

```
SELECT *
```

Always specify columns where practical.

Use transactions for multi-step operations.

Prefer SQLAlchemy ORM.

17. Schema Rules

Never expose ORM models directly.

Always return Pydantic schemas.

18. Async Rules

Use async for:

- Database
- File operations (where beneficial)
- HTTP requests

Avoid unnecessary async for CPU-bound ML tasks.

19. Configuration

Never hardcode.

Use

```
.env
```

Examples

Database URL

Upload Path

API Keys

Report Directory

20. Magic Numbers

Bad

```
if score > 83:
```

Good

```
QUALITY_THRESHOLD = 83
```

21. React Component Rules

One component

↓

One responsibility

Example

```
UploadCard  
  
DatasetTable  
  
KPISection  
  
ChartCard  
  
RecommendationCard
```

Avoid giant pages with embedded UI logic.

22. Component Size

Recommended

```
<200 lines
```

Split if necessary.

23. Props

Always type props.

```
interface KPIProps {  
  
  title: string  
  
  value: number  
  
}
```

24. State

Prefer

Local State

↓

Context

↓

TanStack Query

Avoid unnecessary global state.

25. Styling

Never

```
style={{}}
```

Always use Tailwind utility classes.

No inline colors.

Use design tokens.

26. Forms

Use

React Hook Form

-

Zod

Validation must exist on both frontend and backend.

27. Charts

All charts should:

- Be responsive
 - Exportable
 - Accessible
 - Have legends
 - Show tooltips
-

28. Machine Learning

Every model should:

- Train
- Evaluate
- Compare
- Save metrics

Never expose raw sklearn objects to frontend.

29. Reports

Keep reports concise.

Every report should include:

Summary

↓

Charts

↓

Insights

↓

Recommendations

30. Testing

Backend

pytest

Frontend

Vitest

Every service should have unit tests.

Critical flows should have integration tests.

31. Git Commits

Examples

```
feat(upload): add CSV upload endpoint

fix(cleaning): handle missing values

refactor(profile): split statistics service

docs(api): update report endpoints

test(ml): add regression tests
```

32. Pull Requests

Every PR should answer:

- What changed?
 - Why?
 - Screenshots (if UI)
 - Tests added?
 - Breaking changes?
-

33. Performance

Avoid

Repeated dataframe scans.

Prefer vectorized Pandas operations.

Cache expensive computations when appropriate.

34. Security

Never trust user input.

Validate:

- File type
- File size
- URL format
- API payloads

Sanitize filenames before saving.

35. Documentation

Every module should have:

- Purpose
- Dependencies
- Usage

Complex logic should be documented.

36. AI Development Rules

When AI generates code:

- Follow existing architecture.
 - Never rewrite unrelated files.
 - Never rename APIs without approval.
 - Reuse utilities before creating new ones.
 - Keep changes focused and reviewable.
-

37. Code Review Checklist

Before merging, verify:

- Naming is consistent.
 - No duplicated logic.
 - Types are complete.
 - Errors handled.
 - Logging added.
 - Tests pass.
 - Formatting applied.
 - No unused imports.
 - No commented-out dead code.
-

38. Definition of Production-Ready

A feature is production-ready only if:

- Clean architecture
 - Typed interfaces
 - Validation complete
 - Error handling implemented
 - Logging included
 - Tests written
 - Documentation updated
 - UI responsive
 - No TODOs blocking functionality
-

39. Formatting Tools

Backend

- Ruff
- Black
- isort

Frontend

- ESLint
- Prettier

Formatting is mandatory before commits.

40. Final Principle

"Write code that your future self—and any teammate—can understand without needing an explanation."
